

arXiv Acceptance Helper

현재 구현 상태와 회의록 요구사항을 반영한 설계 검토

DBLP · OpenReview · Official Proceedings · Code Discovery · PDF Download

검토 기준

- 2026-08-06 회의록에 정의된 원래 프로젝트 요구사항
- 현재 29 개 프로젝트 파일로 구현된 Chrome MV3 확장 구조
- 현재 통과한 86 개 테스트와 수동 Chrome 검증이 남아 있는 상태

작성 목적: 직전 검토 답변의 모든 핵심 설명, 판단, 예시, 수정 우선순위를 빠짐없이 문서화.

요약 판단. 회의록의 원래 요구사항과 현재 29 개 파일로 구현된 상태를 같이 보면, 전체 architecture 는 합리적으로 잡혀 있습니다. 다만 중요한 부분 하나가 빠져 있습니다.

현재 구현은 “DBLP + OpenReview 기반 metadata/decision helper”로서는 잘 만들어져 있지만, 회의록에서 가장 강조했던 “공식 proceedings 를 이용한 최종 Track 검증”까지는 아직 도달하지 않았습니다. 따라서 지금 단계에서 가장 먼저 손봐야 할 것은 UI 보다 판정 pipeline 입니다. 회의록에서도 “DBLP 만으로 Main Conference 를 확정하지 않고, 공식 proceedings 를 Track 판정의 최우선 근거로 사용한다”는 원칙을 명시했습니다.

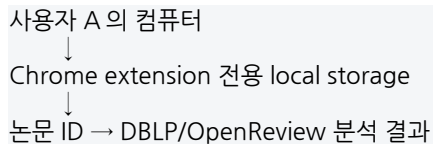
1. 24 시간 cache 는 정확히 어디에 저장되는가?

README 에 적힌 `chrome.storage.local`: 논문별 분석 결과, 저장 시각, 24 시간 TTL 부분이 실제 답입니다.

이건 서버 cache 도 아니고, JavaScript 변수에만 있는 memory cache 도 아닙니다.



즉 다음 구조입니다.



Chrome 공식 문서 기준 `storage.local`은 로컬 머신에 저장되고 extension 을 제거하면 삭제됩니다. 현재 한도는 10MB 입니다.

예를 들어 사용자가 오늘 `1706.03762`를 열어 DBLP/OpenReview 를 조회하면 다음과 같은 정보가 저장됩니다.

```
1706.03762
{
  fetchedAt: 2026-08-25 23:00,
  dblp: ...,
  openreview: ...,
  mergedResult: ...
}
```

2 시간 뒤 다시 열면:

```
Chrome storage 확인
→ 24 시간 안 지남
→ API 요청 안 함
→ cache 사용
```

25 시간 뒤면:

```
cache expired
→ 다시 DBLP/OpenReview 조회
→ 새 결과 저장
```

1.1 Cache 설계에서 권장하는 수정

단순히 `arxivId + timestamp`만 사용하지 말고 다음처럼 만드는 편이 안전합니다.

```
schemaVersion
arxivId
titleFingerprint
authorsFingerprint
fetchedAt
```

```
expiresAt  
result
```

이유는 arXiv 논문 제목이나 저자가 version update 로 바뀌었는데 24 시간 cache 가 남아 있으면 과거 metadata 로 검색할 수 있기 때문입니다.

그리고 UI 에 작게 `Cached 2h ago · expires in 22h` 정도를 보여주면 개발/디버깅에도 상당히 편합니다.

2. PDF 링크는 어떻게 자동으로 아는가?

중요한 점은 “arXiv 화면의 특정 위치가 항상 같아서” 찾는 방식으로 구현하면 안 된다는 것입니다.

현재 구현 설명을 보면 `content.mjs`가 arXiv DOM 에서 PDF 주소를 추출하도록 되어 있는데 이 방식이 맞습니다.

실제 arXiv 페이지에는 보통 `View PDF` 링크가 존재하고, 예를 들어 Attention Is All You Need 의 경우 PDF URL 은 다음과 같은 형태입니다.

```
https://arxiv.org/pdf/1706.03762
```

따라서 가장 안전한 구현은 두 단계입니다.

1 순위
arXiv HTML 에서 실제 PDF `<a href=“...”>`를 찾는다.

↓ 실패하면

2 순위 fallback
현재 arXiv ID 를 이용해
`https://arxiv.org/pdf/{arxivId}`
를 만든다.

즉:

DOM-based URL

↓

있음 → 그대로 사용

없음

↓

arXiv ID 기반 URL 생성

이 방식이면 arXiv 가 버튼의 화면 위치나 CSS 를 바꾸더라도 비교적 안전합니다.

2.1 README 문구 개선

현재 구현 방향은 수정할 필요가 거의 없습니다. 다만 README 의 “논문 HTML 링크와 PDF 링크를 자동 표시합니다.”는 다음과 같이 더 명확하게 표현하는 편이 정확합니다.

arXiv 페이지에 포함된 PDF 링크를 우선 사용하고,
찾지 못할 경우 arXiv ID 기반 PDF URL 을 fallback 으로 사용합니다.

3. PDF 파일명의 “별칭”은 무엇으로 들어가야 하는가?

여기는 현재 README 가 회의록보다 오히려 모호해졌습니다.

회의록에서 최종적으로 합의한 규칙은 다음과 같이 명확합니다.

":"가 있다
→ colon 앞부분만 사용

":"가 없다
→ 논문 제목 전체 사용

마지막에 arXiv ID 추가

예를 들어:

```
StreamingTOM: Memory-Efficient Streaming VideoLLMs ...
```

```
→ StreamingTOM_2501.12345.pdf
```

colon 이 없는 경우:

```
Attention Is All You Need
```

```
→ Attention Is All You Need_1706.03762.pdf
```

따라서 README 의 “제목 별칭과 arXiv ID 로 PDF 파일명을 미리 채운다”라는 표현 자체를 바꾸는 것을 권합니다.

“별칭 탐색”이라고 하면 SAM2, LookAheadKV, StreamingTOM 같은 실제 nickname 을 외부에서 추론하는 것처럼 들리기 때문입니다. 회의록 마지막 원칙도 “별칭이 불확실하면 임의로 생성하지 않는다”였습니다.

V1 에서는 이것을 그냥 `Short title`이라고 정의하는 편이 좋습니다.

```
shortTitle(title):  
  title 에 ":" 존재  
    → 첫 ":" 앞 문자열  
  else  
    → 전체 title
```

그리고 파일명 기본값은 다음처럼 고정합니다.

```
defaultFilename =  
  sanitize(shortTitle) + "_" + arxivId + ".pdf"
```

사용자가 UI 에서 그 값을 직접 수정할 수 있게 두는 현재 설계는 그대로 유지하면 됩니다.

4. GitHub API 검색은 비용이 너무 크지 않은가?

현재 구현 방향은 잘 잡혀 있습니다.

```
논문 열기  
→ GitHub API 호출 안 함
```

```
GitHub additional search 클릭  
→ 그때만 GitHub API 호출
```

4.1 Monetary cost 보다 Rate Limit 이 핵심

GitHub public REST API 를 한두 번 호출한다고 API 사용료가 발생하는 구조는 아닙니다. 문제는 돈보다 Rate Limit 입니다.

GitHub 공식 문서상 unauthenticated REST 요청의 기본 primary limit 은 IP 기준 60 requests/hour 이고, Search API 는 별도의 더 제한적인 rate-limit bucket 을 사용합니다.

따라서 backend 없는 Chrome extension 에서 다음과 같이 논문 페이지를 열 때마다 GitHub Search 를 수행하는 것은 좋지 않습니다.

```
사용자 논문 페이지 열 때마다 GitHub Search
```

현재처럼 다음 우선순위가 적절합니다.

```
HTML scan  
→ PDF scan  
→ 그래도 필요하면 사용자가 Search 버튼
```

4.2 GitHub 검색 정확도에 대한 기대

논문 제목으로 검색한다고 해서 GitHub 검색 결과 1 위가 반드시 official repo 인 것은 아닙니다.

그래서 현재 README의 “결과를 공식 코드가 아닌 검색 후보로 표시”가 매우 중요합니다. 회의록 마지막 결론도 github.io, github.com, huggingface.co 등을 찾는 족족 보여주고 공식 코드라고 함부로 판단하지 말자는 방향으로 정리되어 있습니다.

V1에서는 ‘Official / Likely Official’ 같은 classification 자체를 제거하는 것을 권합니다. 대신 provenance 만 보여주는 편이 더 정확합니다.

```
Paper-linked
PDF에서 직접 발견

arXiv-linked
arXiv HTML에서 직접 발견

Project-linked
project page에서 발견

Search candidate
GitHub 검색으로만 발견
```

예:

```
Code

github.com/SamsungLabs/LookaheadKV
Source: Paper PDF

github.com/user123/lookahead-kv
Source: GitHub search candidate
```

이렇게 하면 프로그램이 공식성까지 판단할 필요가 없습니다.

4.3 GitHub 검색 결과의 짧은 session cache

현재 README에서는 GitHub 결과를 저장하지 않는다고 되어 있습니다. Privacy 측면에서는 깔끔하지만 사용자가 버튼을 여러 번 누르면 같은 Search API를 반복 호출합니다.

따라서 persistent cache 대신 ‘chrome.storage.session’에 GitHub 검색 결과를 30분~1시간 정도 넣는 것도 괜찮습니다.

```
GitHub 클릭
↓
session cache 있음?
├── Yes ──> 사용
└── No  ──> API
```

브라우저를 닫으면 없어지는 cache 이므로 privacy도 거의 그대로 유지할 수 있습니다.

5. OpenReview의 “대화형 challenge”가 대체 무엇인가?

간단히 말하면 OpenReview가 “이 요청이 일반 사용자의 웹브라우저 요청인지 자동화 프로그램 요청인지 확인하겠다”고 중간에 검사 페이지를 보여주는 상황입니다.

정상 API 호출은 다음처럼 JSON이 와야 합니다.

```
extension
↓
OpenReview API
↓
JSON
{
  title: "...",
```

```
}
  decision: "Accept"
}
```

그런데 challenge 에 걸리면 다음과 같이 HTML challenge page, redirect, 429 또는 403 응답이 올 수 있습니다.

```
extension
  ↓
OpenReview
  ↓
잠깐, 이 요청을 확인해야 함
  ↓
HTML challenge page / redirect / 429 / 403
```

실제로 OpenReview 관련 URL 이 `/openreview.net/challenge?redirect=...` 형태로 redirect 되는 사례가 관찰될 수 있습니다.

OpenReview 는 API 2 와 legacy API 1 을 함께 운영하고 있고, 오래된 conference 는 v1 을 사용하는 경우가 있어서 현재 코드의 v1/v2 fallback 자체는 합리적입니다.

6. 브라우저에서 challenge 를 직접 해결했는데 왜 extension API 는 또 실패할 수 있는가?

이 부분을 이해하려면 `origin`과 `cookie`를 알아야 합니다.

사용자가 직접 연 것은 보통 `https://openreview.net/...`입니다. 하지만 extension service worker 는 다음처럼 extension origin 에서 OpenReview API 로 요청합니다.

```
chrome-extension://abcdefgh...
  ↓
https://api2.openreview.net/...
```

브라우저 입장에서는 이들이 전부 동일한 실행 문맥이 아닙니다.

```
openreview.net
api2.openreview.net
chrome-extension://extension-id
```

각각 origin / cookie scope / request context 가 다를 수 있습니다.

Chrome 은 extension 의 third-party 요청에서도 host permission, SameSite, third-party-cookie 정책 등을 적용합니다. 특히 third-party cookie 차단 여부에 따라 cookie 가 전달되지 않을 수 있습니다.

또 일반 `fetch()`의 credential 기본값은 `same-origin`이기 때문에 cross-origin cookie 를 사용하려면 credential 처리도 고려해야 합니다.

그래서 다음 상황이 가능합니다.

```
사용자가 browser tab 에서 challenge 해결
  ↓
openreview.net 에 clearance cookie 생성
  ↓
extension service worker 의
api2.openreview.net fetch
  ↓
그 cookie 가 반드시 적용된다는 보장 없음
```

6.1 왜 브라우저가 이렇게 복잡하게 막는가?

보안 때문입니다.

만약 아무 사이트나 다른 사이트의 로그인 cookie 를 자유롭게 사용할 수 있다면 다음과 같은 공격이 가능해집니다.

```
evil.com
↓
내 Google 로그인 cookie 사용
↓
Google 요청
```

그래서 Same-Origin Policy, SameSite Cookie, third-party cookie restriction 같은 정책으로 사이트 간 credential 전송을 제한합니다.

6.2 OpenReview challenge 처리 권장 설계

challenge 를 우회하려고 노력하지 않는 것을 권합니다.

```
OpenReview API
↓
정상 JSON
↓
자동 분석
↓
OpenReview API
↓
challenge
↓
OpenReview: Manual verification required
↓
[Open OpenReview]
```

여기서 끝내는 것입니다. challenge 해결 후 extension 이 자동 재시도해서 어떻게든 뚫으려고 하는 구조는 불안정합니다.

README 도 현재의 긴 설명보다 다음 정도면 충분합니다.

OpenReview may occasionally block automated requests with an interactive verification page.
In that case, the extension marks OpenReview as unavailable and provides a direct link for manual verification.

7. 지금 구현에서 가장 중요한 문제: Official Proceedings 검증 부재

회의록과 현재 implementation 을 비교하면 다음 차이가 있습니다.

항목	회의록 요구	현재 구현	판단
DBLP	후보 탐색	구현	유지
OpenReview	Decision 검증	구현	유지
Official proceedings	Track 최종 검증	없음	추가 필요
arXiv comment	보조 evidence	명확하지 않음	추가/확인
OpenAlex	fallback	없음	후순위
Crossref	fallback	없음	후순위

항목	회의록 요구	현재 구현	판단
HTML/PDF code	탐색	구현	개선 필요
GitHub Search	마지막 fallback	구현	현재 방식 좋음
Filename	colon 이전 title	표현 모호	명세 수정
Evidence	사용자가 근거 확인	구현	유지

즉 현재 가장 큰 차이는 다음입니다.

회의록

DBLP

↓

candidate venue

↓

Official proceedings

↓

Main / Workshop 검증

반면 현재는 사실상 다음 구조에 가깝습니다.

DBLP

+

OpenReview

↓

결론

8. 이 상태에서 `Verified`라는 표현은 조금 위험하다

예를 들어 DBLP에서 `CVPR 2026` 기록을 찾았다고 합시다. DBLP가 말해주는 건 주로 “이 논문이 CVPR 2026 관련 publication record에 있다”는 것이지 반드시 “CVPR 2026 Main Track”을 의미하지는 않습니다.

회의록에서도 이 점을 명시적으로 경고했습니다.

그래서 confidence를 하나로 뭉치지 않는 것을 권합니다. 현재 `Verified / Probable / Metadata only / Conflicting / Candidate`만 두기보다 내부적으로 다음 세 축을 별도로 관리하는 편이 정확합니다.

Identity

Decision

Track

예:

Identity: Verified

arXiv ID exact match

Decision: Verified

OpenReview official Decision: Accept

Track: Unknown

No official proceedings evidence

UI는 여전히 간단하게 `ICLR 2026 · Accepted / Verified`라고 표시할 수 있지만, 상세 정보에서는 무엇이 verified 됐는지 나눠 보여주는 겁니다.

9. Official Proceedings 를 어떻게 넣을 것인가?

“학회 allowlist 를 만들지 않겠다”와 충돌할 필요가 없습니다. Conference allowlist 와 Source Adapter 는 다른 개념입니다.

예를 들어 다음 정도의 source adapter 를 둘 수 있습니다.

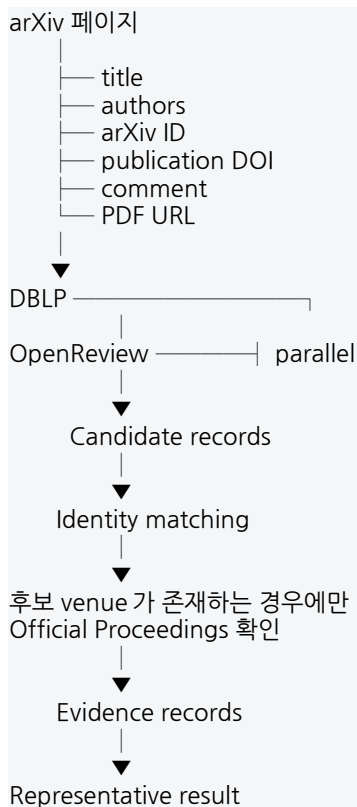
```
proceedings/  
cvf.mjs  
acl-anthology.mjs  
pmlr.mjs  
neurips.mjs  
openreview.mjs
```

회의록에서는 주요 공식 출처를 `CVF Open Access`, `ACL Anthology`, `PMLR`, `NeurIPS Proceedings`, `OpenReview 공식 venue page`, 각 학회 Workshop Proceedings 등으로 정리했습니다.

이건 `CVPR / ECCV / ACL / EMNLP / ICML / NeurIPS ...` 같은 conference allowlist 가 아닙니다. 단지 “이 웹사이트의 데이터를 어떤 식으로 해석할 것인가”를 정의하는 parser 입니다.

10. API 비용까지 생각하면 Proceedings 도 무작정 검색하면 안 된다

추천하는 최종 pipeline 은 다음입니다.



즉 Official Proceedings 를 모든 사이트에 무작정 요청하는 게 아닙니다.

DBLP/OpenReview 에서 후보 발견
→ 필요한 proceedings 만 조회

이렇게 하면 traffic 도 많이 늘지 않습니다.

11. 현재 구현에서 반드시 확인해야 할 문제: DOI

이 부분은 코드 수준에서 반드시 확인하는 것을 권합니다.

arXiv 페이지에는 `10.48550/arXiv.1706.03762` 같은 DOI가 표시될 수 있습니다. 하지만 이것은 conference publication DOI가 아니라 arXiv record의 DataCite DOI입니다.

따라서 내부적으로 단순히 `paper.doi` 하나로 처리하면 위험합니다. 최소한 다음처럼 분리해야 합니다.

```
arxivDoi
publicationDoi
```

예:

```
arxivDoi:
10.48550/arXiv.1706.03762

publicationDoi:
실제 publisher DOI
```

그리고 DBLP matching에서 `publicationDoi exact match`는 강력한 evidence지만 `10.48550/arXiv....`를 publication identifier exact match로 취급하면 안 됩니다.

현재 제공된 프로젝트 요약만으로 `core.mjs`가 이를 구분하는지는 확인할 수 없습니다. 이 부분은 source code에서 바로 검사해야 합니다.

12. Code link scan 도 한 단계 더 보강하는 것이 좋다

현재 구조는 `arXiv HTML → PDF annotation → PDF visible text → GitHub search`라서 방향은 맞습니다.

하지만 단순히 `github.com`, `huggingface.co`라는 이유만으로 전부 가져오면 false positive가 발생할 수 있습니다.

예를 들어 arXiv 페이지 자체에는 arXivLabs의 일반적인 `Hugging Face`, `CatalyzeX`, `DagsHub` 링크가 존재할 수 있습니다. 이것은 해당 paper author가 제공한 repository가 아닐 수 있습니다.

따라서 HTML link parser에서 다음을 구분해야 합니다.

```
arXivLabs generic link
```

```
vs.
```

```
paper-specific external link
```

PDF에서도 References에 다른 논문의 GitHub URL이 나올 수 있습니다. 그래서 URL만 보지 말고 context를 같이 보는 것이 좋습니다.

```
강한 evidence
```

```
"We release our code at ..."
```

```
"Project page:"
```

```
"Code:"
```

```
첫 페이지 저자 아래 URL
```

```
중간 evidence
```

```
Appendix / Acknowledgement의 repository
```

```
약한 evidence
```

```
References 안의 GitHub URL
```

이것을 Official 판정에 쓰지 않더라도 `Source: PDF page 1 · near "Code"`처럼 provenance를 보여주면 매우 좋습니다.

13. 회의록의 “server message” 개념을 현재 architecture 기준으로 정리

회의록 첫 부분에는 “API 를 요청해서 가져오는 느낌이 아니라 server message 를 웹에 보내고 응답을 가져오는 느낌”이라는 이해가 있었는데, 현재 구현 architecture 를 보면 이제 정확히 구분할 수 있습니다.

```
arXiv webpage
|
content.mjs
|
chrome.runtime.sendMessage(...)
|
service-worker.mjs
|
fetch(...)
▼
DBLP / OpenReview / GitHub
```

여기서 `content → service worker`는 Chrome extension 내부 message 입니다. 서버에 보내는 message 가 아닙니다.

실제 서버 통신은 `service worker → DBLP/OpenReview`의 HTTP `fetch()`입니다.

Chrome MV3에서는 cross-origin fetch 를 service worker 에서 수행하고 필요한 domain 을 `host\_permissions`에 선언하는 구조가 일반적입니다. 즉 현재 architecture 자체는 정석적인 MV3 구조에 가깝습니다.

14. 현재 29 개 파일 구조에서 수정할 우선순위

P0 - 판정 정확성

- Official Proceedings verification 을 추가하거나, 추가 전까지 Track 관련 Verified 표현을 약화해야 합니다.
- DOI 를 arXiv DOI / publication DOI 로 분리하는지 확인해야 합니다.

이 두 가지가 가장 중요합니다.

P1 - 의미 명확화

- `alias`를 실제 별칭 탐색처럼 표현하지 말고 `short title / filename title`로 바꿉니다.
- 규칙은 colon 이전, 없으면 full title, 마지막에 arXiv ID 를 붙이는 것으로 고정합니다.
- OpenReview challenge 설명을 단순화합니다.

P1 - Code provenance

- `Official Code / Likely Official` 추론보다는 `Paper-linked / Project-linked / Search candidate` 방식으로 바꾸는 것을 권합니다.

P2 - 비용/UX

- GitHub 결과는 `storage.session`으로 짧게 cache 하는 것을 고려합니다.
- DBLP/OpenReview 분석 cache 에는 source, fetchedAt, expiresAt, schemaVersion, metadata fingerprint 를 넣습니다.

P2 - False Positive 방지

- arXiv Labs 의 generic Hugging Face/CatalyzeX 링크와 References 속 repo 를 paper code 로 오인하지 않는 테스트를 추가합니다.

15. 86 개 unit test 가 모두 통과했다는 것과 “실제로 잘 작동한다”는 것은 별개

현재 확인된 `86 tests PASS / lint PASS / manifest validation PASS`는 코드 내부 계약이 맞다는 것을 의미합니다.

하지만 현재 보고서에서도 실행하지 못했다고 한 다음 항목들은 mock test 로 완전히 검증할 수 없습니다.

Shadow DOM 실제 위치
extension service-worker CORS
OpenReview challenge
chrome.storage 실제 cache
Downloads API
Save As dialog
PDF.js worker

따라서 다음 개발 단계는 기능 추가만 하는 것이 아니라 실제 Chrome 에서 최소한 다음 흐름을 검증하는 것이 우선입니다.

arXiv page
↓
버튼 표시
↓
Network tab 확인
↓
버튼 열기 전 API = 0
↓
첫 클릭
↓
DBLP/OpenReview 요청
↓
storage.local 확인
↓
새로고침
↓
API = 0, cache 사용
↓
Refresh
↓
API 재요청
↓
PDF scan
↓
Download

16. 최종 판단

architecture 를 알아엮을 필요는 없습니다.

content script
↓
message
↓
service worker
↓
sources
↓
core
↓
storage

라는 현재 모듈 분리는 유지하면 됩니다.

다만 회의록의 핵심 요구사항과 현재 implementation 사이에는 다음 한 줄짜리 차이가 있습니다.

현재:
DBLP + OpenReview 로 "논문이 어디에 존재하는가"를 꽤 잘 판단

목표:
DBLP + OpenReview 로 후보를 찾고
Official Proceedings 로 "정말 어느 Track 에 publish 됐는가"까지 검증

따라서 다음 구현 순서는 `Official Proceedings → DOI 구분 → Code provenance 개선 → GitHub session cache → 실제 Chrome 검증` 순으로 잡는 것이 가장 합리적입니다. 이 순서로 가면 회의록에서 정의했던 “단순 acceptance label 이 아니라, 공식 근거를 추적 가능한 형태로 보여주는 extension”이라는 원래 목적과 현재 구현이 다시 정확히 맞춰집니다.

17. 참고 출처와 근거

아래 링크들은 직전 검토 답변에서 근거로 사용한 공개 문서입니다. 회의록에서 파생된 설계 원칙은 별도로 2026-08-06 회의록을 기준으로 했습니다.

- [Chrome Extensions - Storage API](https://developer.chrome.com/docs/extensions/reference/api/storage) - <https://developer.chrome.com/docs/extensions/reference/api/storage>
- [arXiv - Attention Is All You Need](https://arxiv.org/abs/1706.03762) - <https://arxiv.org/abs/1706.03762>
- [GitHub REST API - Rate limits](https://docs.github.com/en/rest/using-the-rest-api/rate-limits-for-the-rest-api) - <https://docs.github.com/en/rest/using-the-rest-api/rate-limits-for-the-rest-api>
- [OpenReview API documentation](https://docs.openreview.net/getting-started/using-the-api) - <https://docs.openreview.net/getting-started/using-the-api>
- [Chrome Extensions - Storage and Cookies](https://developer.chrome.com/docs/extensions/develop/concepts/storage-and-cookies) - <https://developer.chrome.com/docs/extensions/develop/concepts/storage-and-cookies>
- [MDN - Using the Fetch API](https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch) - https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch
- [Chrome Extensions MV3 - Security](https://developer.chrome.com/docs/extensions/mv3/security) - <https://developer.chrome.com/docs/extensions/mv3/security>
- [OpenReview challenge redirect 사례 참고](https://www.ablesci.com/assist/detail?id=Qpm6jV) - <https://www.ablesci.com/assist/detail?id=Qpm6jV>

회의록 근거 메모

2026-08-06 회의록에서 특히 중요한 원칙: DBLP 만으로 Main Conference 를 확정하지 않음, 공식 proceedings 를 Track 판정의 핵심 근거로 사용, Main/Workshop 이 함께 발견되면 Main 을 대표로 하되 기록을 모두 유지, 출처 충돌과 불완전성을 공개, 별칭이 불확실하면 임의 생성하지 않음, 검색으로 발견된 GitHub 저장소를 자동으로 공식 코드로 분류하지 않음.